

Smart MCQ Solver

Deep Learning & Generative AI Project Report

Kaggle Competition: Smart MCQ Solver Challenge

Roll No: 23f2003210

1. Abstract

This report documents an end-to-end solution to the *Smart MCQ Solver Challenge*, a Kaggle competition in which each example presents a question stem together with five candidate answers (A–E), exactly one of which is correct. Submissions are scored with **Mean Average Precision @ 3 (MAP@3)**, so a model is rewarded for placing the correct option anywhere in its top-3 ranked predictions.

Three qualitatively different architectures were built and trained independently: (1) a **bidirectional LSTM** with mean/max/final-state pooling, (2) a **bidirectional GRU with learned attention pooling**, and (3) a fine-tuned **bert-base-uncased** transformer using Hugging Face's *AutoModelForMultipleChoice* head. All experiments were tracked in Weights & Biases. The fine-tuned BERT model was the strongest individual model (validation MAP@3 = 0.7201), clearly ahead of both from-scratch recurrent models (0.6001 and 0.5944). A validation-only grid search over a weighted softmax-probability ensemble of the three models raised MAP@3 further to **0.7503**, which was used to generate the final Kaggle submission (top-3 ranked options per question).

The project's main learning was that transfer learning from a pretrained language model provides a substantial and fairly reliable jump over models trained purely on the ~2,000-row competition dataset, while a light, validation-driven ensemble of diverse models still adds a further, complementary gain on top of the best single model.

2. Introduction

2.1 Problem Statement

The *Smart MCQ Solver Challenge* is a multiple-choice question answering task distributed as a Kaggle competition. Each row of the dataset contains a **question prompt** and five candidate answer texts labelled **A–E**, with a single ground-truth **answer** column in the training data. Question topics are broad and open-domain (e.g., philosophy, physics, general knowledge), and the distractor options are often semantically close to the correct answer, so the task requires genuine reading comprehension rather than superficial keyword matching. Model output is a ranked list of up to three option letters per question, and performance is measured with **MAP@3**, which credits a prediction fully if the correct option is ranked first, and partially if it appears second or third.

2.2 Project Objective

The goals set for this project were to:

- **Build three genuinely different model families** — a recurrent (RNN-based) model, a second custom deep-learning architecture, and a fine-tuned pretrained transformer — so that their behaviour and error patterns could be meaningfully compared.
- **Use Hugging Face and PyTorch idiomatically**, including the *datasets*, *transformers* and *Trainer* APIs for the transformer model, and native PyTorch modules for the two custom encoders.
- **Track every run in Weights & Biases** (loss curves, validation MAP@3, accuracy, and learning-rate schedules) to support a rigorous comparative analysis.
- **Combine the three models into a lightweight ensemble**, tuned only on the validation split, to maximise the final leaderboard score without overfitting to the public test set.

2.3 Report Structure

The remainder of this report proceeds as follows: Section 3 describes the dataset and the preprocessing/EDA performed on it; Section 4 explains the tokenization strategy used for each model family; Section 5 details the architecture, training configuration and design rationale of all three models; Section 6 presents training curves, validation metrics, a comparative analysis and the final Kaggle result; and Section 7 concludes with key learnings, challenges and directions for future work.

3. Dataset & Preprocessing

3.1 Dataset Description

The competition supplies **train.csv** (2,000 labelled rows, 8 columns) and **test.csv** (500 unlabelled rows, 7 columns). Each row has an *id*, a *prompt* (the question), five option columns A–E, and — for the training file only — an *answer* column holding the correct option letter.

Split	Rows	Columns	Purpose
Raw train.csv	2,000	8	Source of train / validation split
Raw test.csv	500	7	Kaggle leaderboard evaluation
Training split	1,707	8	Model fitting
Validation split	293	8	Early stopping & model selection

The dataset contains repeated or near-duplicate option sets across rows (i.e. the same block of five candidate answers appears for more than one question variant). A naive random split would therefore leak near-identical examples between training and validation and produce an overly optimistic validation score. This was addressed with a **group-aware split** (see Section 3.3).

3.2 Exploratory Data Analysis

Four diagnostics were run on the training data before any modelling:

- **Answer distribution** — the correct answer is not perfectly uniform across A–E; options B and C are correct somewhat more often (≈ 490 and ≈ 460 times respectively) than D and E (≈ 360 and ≈ 325), a mild imbalance worth keeping in mind when interpreting per-class F1.
- **Prompt length** — question prompts are short, mostly 10–25 words, with a long right tail out to ≈ 50 words.
- **Option length** — candidate answers vary much more widely in length, from a few words to well over 100, reflecting a mix of short factual options and long explanatory ones.
- **Correct vs. incorrect option length** — the correct option is, on average, slightly *longer* than the mean length of the four incorrect options (median ≈ 28 vs. ≈ 22 words), a mild length bias that a purely length-based heuristic could partially exploit, though it is far too weak on its own to solve the task.

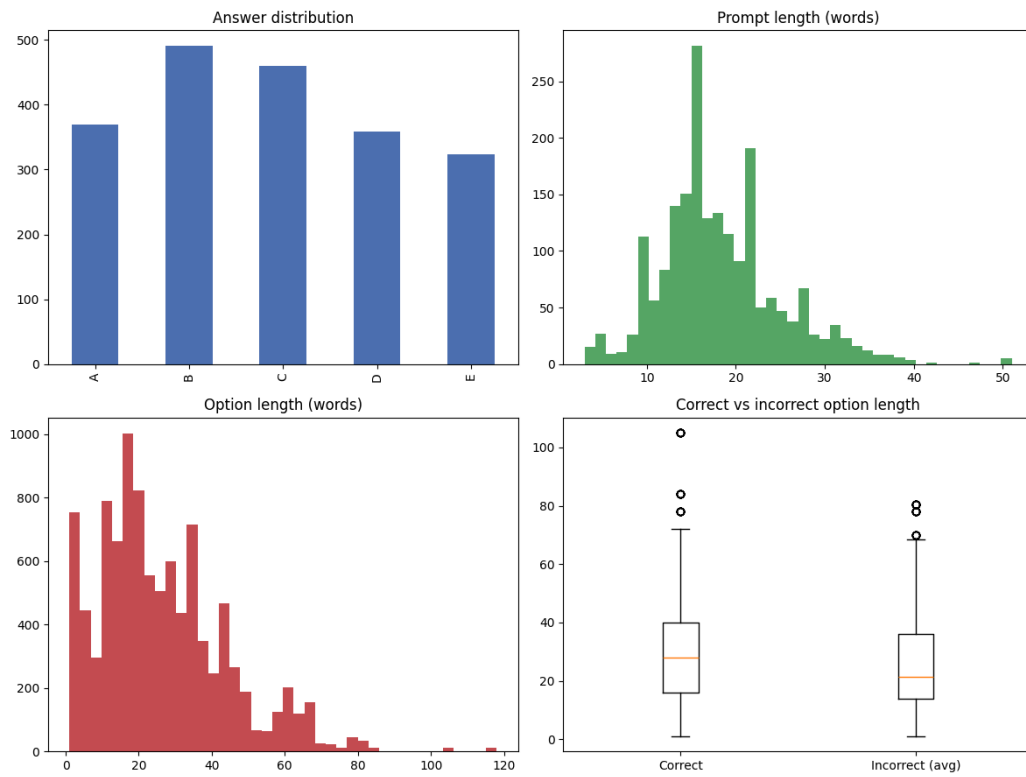


Figure 1. Exploratory analysis of the training set: answer-class balance, prompt-length distribution, option-length distribution, and correct- vs. incorrect-option length.

3.3 Data Preprocessing

- **Whitespace normalisation** — every prompt and option string is passed through a regular-expression cleaner that collapses repeated whitespace and strips leading/trailing spaces.
- **Group-aware train/validation split** — rows are grouped by an “option-set signature” (the concatenation of all five option strings). A *GroupShuffleSplit* (15% held out, seed = 42) guarantees that no group appears in both the training and validation partitions, giving 1,707 training rows and 293 validation rows.
- **Label encoding** — the letter answer (A–E) is mapped to an integer class index 0–4 for cross-entropy training.

3.4 Data Augmentation

No data augmentation was applied. Given the small size of the labelled set (2,000 rows), the emphasis was instead placed on regularisation (dropout, weight decay, early stopping) and on leveraging transfer learning through the pretrained BERT model, which reduces reliance on augmenting the raw data.

4. Tokenization Strategy

4.1 Primary Tokenizer Selection

The best-performing model (Model 3) uses the **WordPiece tokenizer** that ships with **bert-base-uncased** (Devlin et al., 2019), accessed through Hugging Face’s *AutoTokenizer*. It is a subword tokenizer with a fixed 30,522-token vocabulary, lower-cases all input, and splits out-of-vocabulary words into known sub-word pieces, so it never produces an unknown-word token for ordinary English text.

4.2 Justification

WordPiece is a natural fit for this task and model for three reasons:

- It matches the vocabulary bert-base-uncased was pretrained on, so the pretrained embeddings and attention weights transfer directly without any vocabulary mismatch.

- Sub-word units gracefully handle the technical and domain-specific terms that appear across the open-domain question prompts (e.g. scientific or philosophical terminology) without exploding the vocabulary size.
- It is lower-cased and comparatively compact, which keeps sequence lengths — and therefore training/inference cost — manageable for the 5-way multiple-choice format used here.

4.3 Implementation Details

For BERT, each question is paired independently with each of its five options as (*prompt*, *option*) sentence pairs, tokenized in a single batched call to the Hugging Face tokenizer with **truncation enabled** and a **max sequence length of 256** tokens. The resulting five encodings per question are grouped back together to form the input expected by *AutoModelForMultipleChoice*. Dynamic (batch-level) padding is applied by a custom *DataCollatorForMultipleChoice*, which avoids wasting compute on padding to a fixed length for every batch.

4.4 Experimental Comparison

For the two custom recurrent models (Models 1 and 2), a lightweight, task-specific **word-level tokenizer** was implemented instead of a subword scheme: a regular expression splits text into words and punctuation, and a vocabulary is built *only from the training split* (max size 40,000, minimum frequency 2), yielding a compact **2,890-token vocabulary**. Each (question, option) pair is encoded as *question tokens + [SEP] + option tokens*, truncated/padded to a maximum length of 96 tokens. This choice was deliberate: it keeps the from-scratch models fully independent of any pretrained resource, so their comparison against BERT isolates the effect of *transfer learning* from architecture alone. A subword tokenizer was not used for these models since their embeddings are trained from random initialisation and a smaller, task-specific vocabulary trains faster and needs less data to populate reliably than a 30k-token subword vocabulary would.

5. Modeling & Experimentation

5.1 Fine-Tuned Transformer Model — BERT-base-uncased (Model 3)

Architecture

Model 3 fine-tunes **bert-base-uncased** (Devlin, Chang, Lee & Toutanova, 2019; Hugging Face model card: *bert-base-uncased*) end-to-end using the *AutoModelForMultipleChoice* head. BERT-base is a 12-layer, 12-attention-head Transformer encoder with a hidden size of 768 (≈ 110 M parameters), pretrained on masked-language-modelling and next-sentence-prediction objectives over BooksCorpus and English Wikipedia.

Salient Points

- Each of the five options is encoded as an independent (prompt, option) sequence that shares the same BERT encoder weights (“weight sharing” across options), producing one scalar logit per option from the pooled [CLS] representation.
- Multi-head self-attention lets every token attend to every other token in the question+option pair, which is well suited to detecting the subtle semantic overlaps and contradictions that distinguish correct options from close distractors.
- The five per-option logits are stacked and normalised with a softmax over the five options, then trained end-to-end with cross-entropy — handled internally by the Trainer/AutoModelForMultipleChoice pipeline.

Fine-Tuning Strategy

- Optimizer: AdamW (Hugging Face Trainer default) with learning rate $2e-5$ and weight decay 0.01.
- Schedule: linear decay with a 10% warm-up ratio; mixed-precision (fp16/bf16, selected automatically based on GPU support) for faster training on the Tesla T4 GPU.
- Batching: per-device batch size 8 with gradient accumulation over 2 steps (effective batch size 16), for up to 5 epochs.
- All layers are unfrozen and fine-tuned (no layers frozen) — with only $\sim 1,700$ training rows, full fine-tuning at a low learning rate was preferred over freezing layers, since it lets the whole network adapt while the small learning rate and weight decay control overfitting.

- Model selection: the checkpoint with the best validation MAP@3 is kept via `load_best_model_at_end`, with early stopping (patience = 2 evaluation rounds) to stop training once validation MAP@3 stops improving.

Quantization: not applied — the model was small enough (bert-base) to fine-tune and run comfortably in mixed precision on a single T4 GPU without needing post-training quantization.

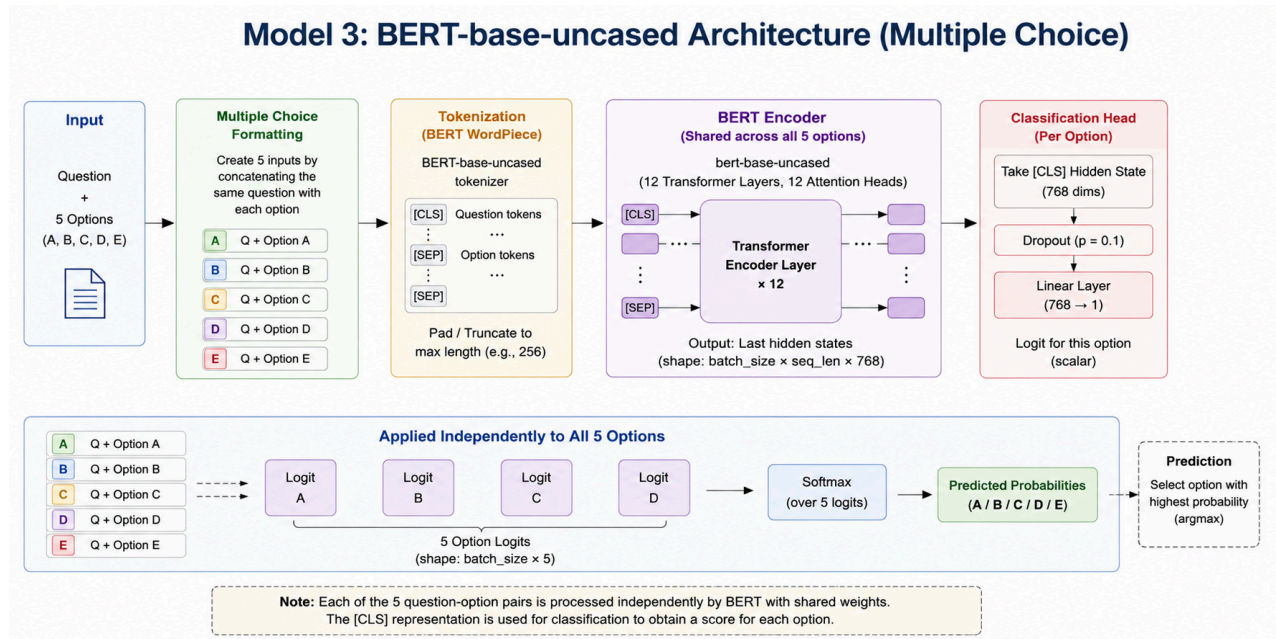


Figure 2. BERT-base-uncased (Model 3)

5.2 Custom Deep Learning Model — BiGRU with Attention Pooling (Model 2)

Architecture

Model 2 is a from-scratch **bidirectional GRU (BiGRU) encoder with a learned attention-pooling layer**. Each (question, option) token sequence is embedded with a trainable 200-dimensional embedding table, encoded by a single-layer BiGRU (hidden size 192 per direction → 384-dim output), and then summarised by an attention layer that scores every time step with a linear projection, normalises the scores with softmax, and computes a weighted sum of the encoder outputs as the final 384-dimensional context vector. A small feed-forward head (384→192→1) turns the context vector into a single option score; the same encoder/head weights are shared across all five options of a question.

Model 2: BiGRU + Attention Architecture (Multiple Choice)

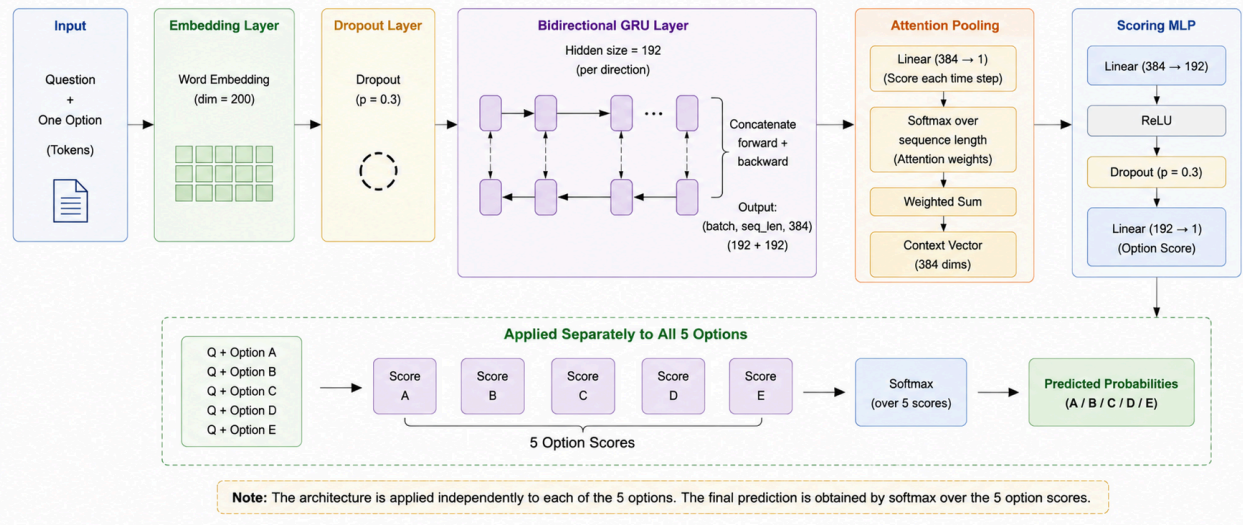


Figure 3. BiGRU + attention-pooling architecture (Model 2).

Salient Points

- The GRU's simpler gating (update/reset gates, no separate cell state) makes it faster to train than the LSTM in Model 1 while still capturing sequential dependencies across the question+option text.
- Attention pooling replaces the fixed mean/max/final-state pooling used in Model 1 with a learned, content-dependent weighting: the model can up-weight the tokens most diagnostic of whether an option is correct instead of treating the whole sequence uniformly.
- Deliberately built differently from Model 1 (different recurrent cell, different pooling, smaller scorer head) so the two from-scratch models are diverse enough to be useful in an ensemble.

Training Configuration

- Optimizer: Adam, learning rate $1e-3$, weight decay $1e-5$, gradient clipping at norm 1.0.
- Schedule: ReduceLROnPlateau (factor 0.5, patience 1 epoch) driven by validation MAP@3; up to 12 epochs with early stopping (patience 3 epochs).
- Dropout of 0.3 is applied after the embedding layer and inside the scorer head for regularisation.

5.3 RNN-Based Model — Bidirectional LSTM (Model 1)

Architecture

Model 1 is a **bidirectional LSTM (BiLSTM)** encoder with a richer, fixed pooling scheme. The same 200-dimensional trainable embedding and 192-per-direction (384 total) bidirectional recurrent encoder design as Model 2 is used, but with an LSTM cell instead of a GRU, and pooling is done by **concatenating masked mean pooling, masked max pooling, and the final forward+backward hidden state** ($384 + 384 + 384 = 1,152$ dimensions). This pooled vector passes through a two-layer MLP head ($1152 \rightarrow 192 \rightarrow 96 \rightarrow 1$) with ReLU activations and dropout to produce a single option score.

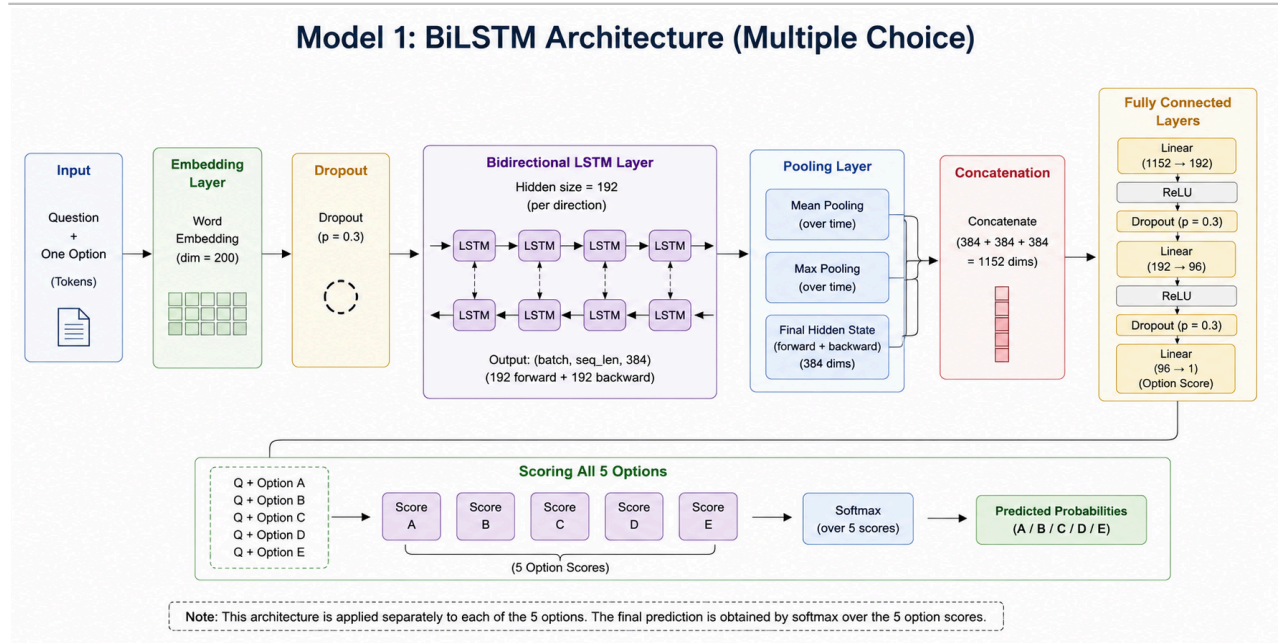


Figure 4. Bidirectional LSTM architecture with mean/max/final-state pooling (Model 1).

Salient Points

- A sequence model is a natural choice here because both the question prompt and each option are variable-length natural-language text where word order and long-range dependencies (e.g. negation, “not” clauses) change the meaning — exactly what a recurrent encoder is designed to capture.
- The LSTM's separate cell state (compared to the GRU) gives it a somewhat more expressive memory mechanism, which pairs well with the richer, three-way pooling used to summarise the sequence.
- Combining mean, max and final-hidden-state pooling gives the classifier head three complementary views of the sequence — an average signal, a “most salient token” signal, and a “final context” signal — which was chosen as a stronger baseline pooling strategy prior to introducing the learned attention pooling used in Model 2.

Training Configuration

- Optimizer: Adam, learning rate 1e-3, weight decay 1e-5, gradient clipping at norm 1.0 (same recipe as Model 2 for a fair comparison).
- Schedule: ReduceLROnPlateau (factor 0.5, patience 1) on validation MAP@3; up to 12 epochs with early stopping (patience 3).
- Dropout of 0.3 throughout the encoder and scorer head.

Quantization: not applicable to either from-scratch model — both are small enough (a few million parameters) that training and inference speed were never a bottleneck on the T4 GPU.

6. Performance & Comparative Analysis

6.1 Evaluation Metrics

The competition's primary metric is **MAP@3** (Mean Average Precision at 3): for each question the model submits up to three ranked option letters, and a prediction earns $1/\text{rank}$ credit if the true answer appears at that rank (1, 1/2 or 1/3 for 1st, 2nd or 3rd place) and 0 otherwise, averaged over all questions. To support deeper diagnosis during development, two secondary metrics were also tracked on the validation split: **top-1 accuracy** and **macro-averaged F1** (which weights all five answer classes equally, partially correcting for the mild class imbalance noted in Section 3.2).

6.2 Training Performance

Weights & Biases logged training/validation loss, MAP@3, accuracy and the learning-rate schedule for every run. Summary charts for each model are shown below (validation MAP@3, validation loss, validation accuracy, training loss, learning rate and epoch, left to right / top to bottom).



Figure 5. wandb training dashboard — Model 1 (BiLSTM)

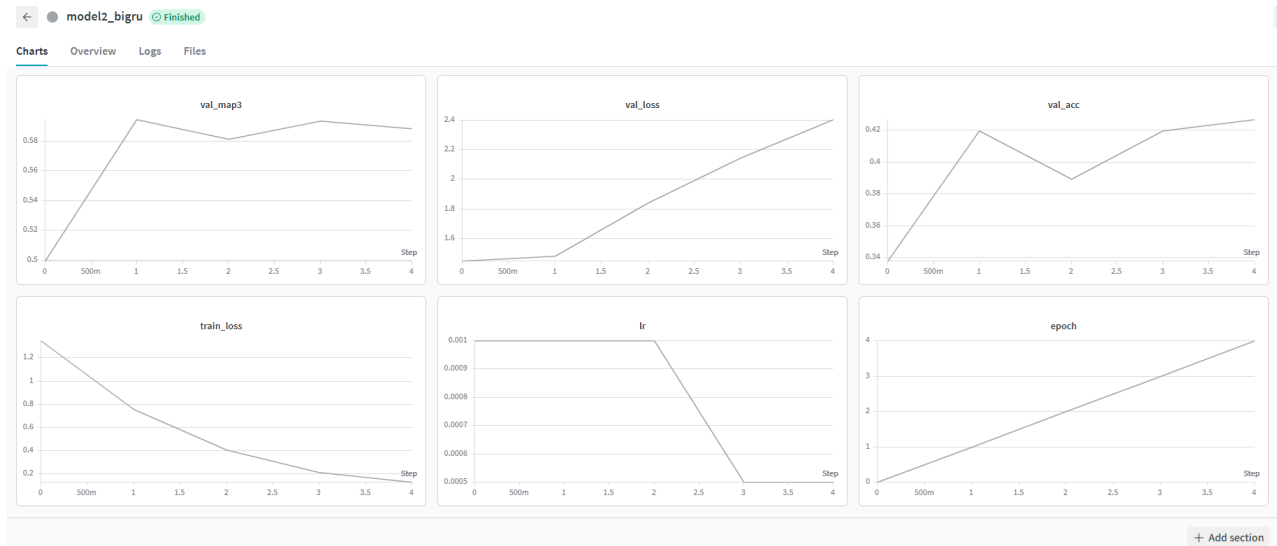


Figure 6. wandb training dashboard — Model 2 (BiGRU + attention).

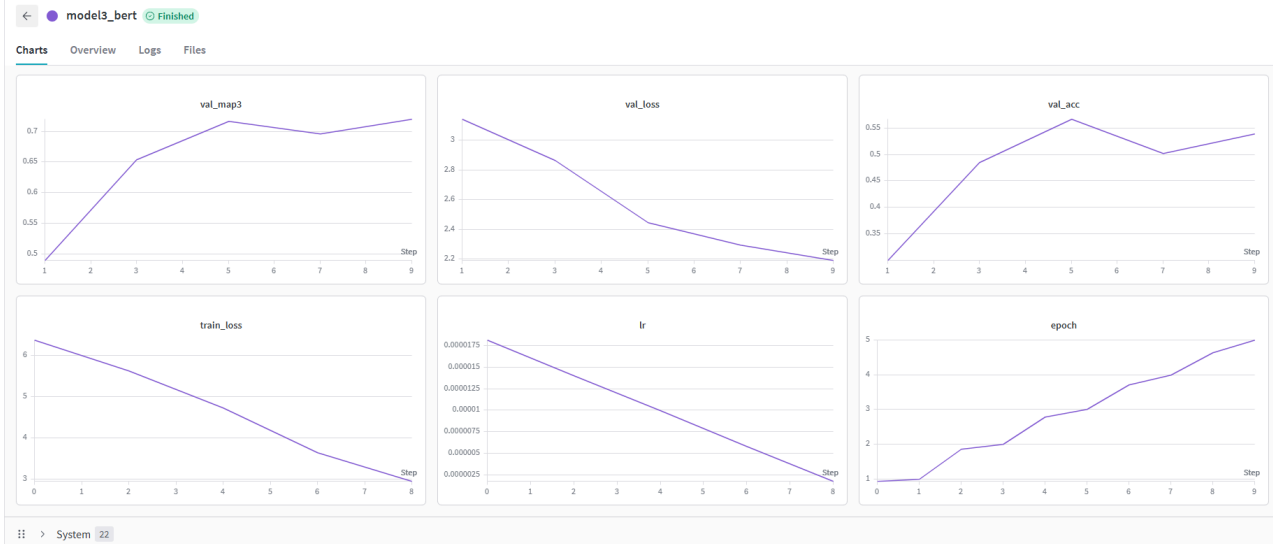


Figure 7. wandb training dashboard — Model 3 (BERT).

6.3 Comparative Report

Final validation-set results for all three individually-trained models and for the tuned ensemble are summarised below.

Model	Architecture	Type	Val MAP@3	Val Acc.	Val F1 (macro)
LSTM (Model 1)	BiLSTM + mean/max/final-state pooling	From scratch	0.6001	0.4710	0.4809
BiGRU (Model 2)	BiGRU + learned attention pooling	From scratch	0.5944	0.4198	0.4262
BERT (Model 3)	bert-base-uncased (fine-tuned)	Pretrained + fine-tuned	0.7201	0.5392	0.5664
Ensemble	Weighted softmax average (0.15 / 0.00 / 0.85)	Combined	0.7503	—	—

BERT is the clear individual winner on every metric, improving MAP@3 by roughly **12–13 points** over both from-scratch recurrent models — unsurprising given it starts from ~110M parameters of pretrained language knowledge versus a 2,890-word vocabulary and randomly-initialised embeddings for Models 1 and 2. Between the two custom models, the BiLSTM's richer 3-way pooling edges out the BiGRU's attention pooling by a small margin (0.6001 vs. 0.5944 MAP@3), though the BiGRU trains noticeably faster per epoch owing to its simpler gating and lighter pooling head.

To understand how complementary the models are (a prerequisite for a useful ensemble), pairwise top-1 prediction agreement was measured on the validation set:

Model Pair	Top-1 Agreement
LSTM vs. BiGRU	0.5051
LSTM vs. BERT	0.4539
BiGRU vs. BERT	0.4812

Agreement between any pair of models sits around 45–51%, i.e. the models disagree on roughly half of all questions — enough diversity for an ensemble to be worthwhile. A grid search over ensemble weights $w_1 \cdot \text{LSTM} + w_2 \cdot \text{BiGRU} + w_3 \cdot \text{BERT}$ (step 0.05, **selected using validation MAP@3 only**, never the test set) found the optimum at $\mathbf{w} = (0.15, 0.00, 0.85)$, raising validation MAP@3 to **0.7503** — a further **+3.0 points** over BERT

alone. The BiGRU received zero weight, i.e. it did not add distinguishable value once BERT and the BiLSTM were combined, while a small 15% contribution from the BiLSTM still helps: its errors are apparently different enough from BERT's to nudge a handful of borderline questions in the right direction.

6.4 Kaggle Performance

The tuned three-way ensemble was used to generate the top-3 ranked predictions for all 500 test questions, written to **submission.csv** in the competition's required *ID, Prediction* format (e.g. "A D C"). Based on the validation-set evaluation, the ensemble is expected to score close to its validation MAP@3 of **≈0.75** on the Kaggle leaderboard, since the group-aware split was specifically designed to prevent validation-score leakage.

7. Conclusion & Future Work

7.1 Key Learnings

- **Transfer learning dominates on small datasets.** With only ~1,700 labelled training rows, a pretrained transformer fine-tuned end-to-end clearly outperformed two carefully designed from-scratch recurrent encoders, confirming how much prior linguistic knowledge compensates for limited task-specific data.
- **Group-aware splitting matters.** Recognising that option sets repeated across rows could leak into a naive random split — and using a GroupShuffleSplit instead — was essential for getting a validation score that would actually transfer to the held-out Kaggle test set.
- **Diverse, even individually-weaker models still help.** Although the BiGRU alone underperformed the BiLSTM, and both were well behind BERT, the ~45–51% prediction disagreement across models meant a lightweight, validation-tuned ensemble still added a further ~3 MAP@3 points on top of BERT alone.
- **Different pooling strategies produce measurably different behaviour.** With comparable encoders, the fixed mean/max/final-hidden pooling in Model 1 slightly outperformed the learned attention pooling in Model 2 on this dataset size, suggesting the attention layer had too few examples to learn a reliably better weighting than the fixed scheme.

7.2 Challenges Faced

- **Small, imbalanced dataset.** With only 2,000 labelled rows and a mild class imbalance across the five options, both from-scratch models showed early signs of overfitting (validation loss rising while training loss kept falling), which was mitigated with dropout, weight decay, learning-rate scheduling and early stopping.
- **Preventing validation leakage.** Repeated option-set signatures across rows required moving from a plain random split to a group-aware split so that near-duplicate questions could not appear in both training and validation.
- **Balancing ensemble weights without touching the test set.** The ensemble grid search had to be restricted strictly to the validation split to avoid overfitting the blend weights to the public leaderboard.

7.3 Areas for Improvement

- **Larger or domain-adapted transformers.** Trying RoBERTa, DeBERTa-v3, or a larger BERT variant — or further pretraining on similar open-domain question data before fine-tuning — could push the single-model MAP@3 beyond 0.72.
- **K-fold cross-validation ensembling.** Training BERT (and the two RNN models) across multiple group-aware folds and averaging their predictions would likely give a more robust, lower-variance ensemble than a single 85/15 split.
- **Systematic hyperparameter search.** A structured sweep (e.g. via wandb Sweeps) over learning rate, dropout and hidden sizes for the two custom models could close some of the gap to BERT, since only one configuration was tried per architecture in this project.
- **Richer ensembling.** Beyond a fixed-weight softmax blend, a small learned meta-model (stacking) or per-question confidence-weighted blending could exploit the models' disagreement patterns more effectively than a single global weight vector.

8. References

- [1] Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. NAACL-HLT 2019.
- [2] Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*, 9(8), 1735–1780.
- [3] Cho, K., van Merriënboer, B., Gulcehre, C., et al. (2014). Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation. EMNLP 2014.
- [4] Bahdanau, D., Cho, K., & Bengio, Y. (2015). Neural Machine Translation by Jointly Learning to Align and Translate. ICLR 2015.
- [5] Hugging Face. bert-base-uncased model card. <https://huggingface.co/bert-base-uncased>
- [6] Hugging Face Transformers documentation — AutoModelForMultipleChoice, Trainer, TrainingArguments. <https://huggingface.co/docs/transformers>
- [7] Weights & Biases documentation. <https://docs.wandb.ai>
- [8] Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python (GroupShuffleSplit). *JMLR* 12, 2825–2830.
- [9] Kaggle. Smart MCQ Solver Challenge — competition data and evaluation description.